Resumo prova 2 Grafos

Matéria:

+ caminho crítico
+ excentricidade
+ Kosaraju

Conectividade em Grafos Direcionados

- **Fracamente Conexo:** Existe um caminho entre quaisquer dois vértices se as arestas forem consideradas não direcionadas. (Busca por Largura / Profundidade)
- **Semi-Fortemente Conexo:** Para qualquer par de vértices u e v, existe um caminho unidirecional, ou de u para v ou de v para u. (Dij Ketra / Bellmanford)
- **Fortemente Conexo:** Existe um caminho em ambas as direções entre quaisquer dois vértices do grafo. (Kosaraju)

Algoritmo de Kosaraju

- Encontra componentes fortemente conexos.
- Utiliza duas buscas em profundidade (DFS): uma para obter a ordem topológica e uma para cada componente fortemente conexo no grafo transposto.

Algoritmos de Caminho Mínimo (Shortest Path)

- **Dijkstra:** Encontra o caminho mais curto de um vértice para todos os outros (não funciona com arestas negativas).
- **Bellman-Ford:** Resolve o problema de caminho mínimo, mesmo com arestas negativas, mas com maior complexidade.

Algoritmos de Fluxo (Mínimo e Máximo)

- **Fluxo Min-Max:** Encontra o caminho que maximiza o fluxo enquanto minimiza gargalos.
- **Fluxo Max-Min**: Encontra o caminho que minimiza o fluxo máximo.

Caminho Máximo (Longest Path)

- Ordenação **Topológica por largura** (ideia de pegar as **bases** do grafo): Utilizada para encontrar o caminho mais longo em DAGs (grafos direcionados acíclicos).

Árvores Geradoras Mínimas / Máximas (Minimum/Maximum Spanning Trees)

- **Prim:** Expande uma árvore começando por um vértice e adicionando arestas de menor custo.
- **Kruskal:** Adiciona as arestas de menor custo sem formar ciclos, até conectar todos os vértices.

Algoritmo de Johnson

- Usa Dijkstra repetidamente para todos os pares de vértices, mesmo com arestas negativas, ao transformar pesos negativos em não negativos.

~~Algoritmo de Floyd-Warshall~~

- ~~Encontra o caminho mínimo entre todos os pares de vértices. (Ainda a ser estudado).~~

Conectividade em Grafos Direcionados

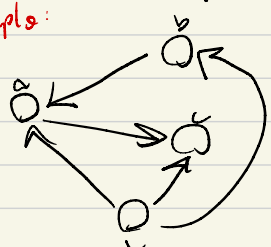
▷ **Fracamente conexo**: Quando existe um caminho entre quaisquer dois vértices ao ignorarmos a direção das arestas. Para identificarmos esse tipo de grafo, podemos:

1- criar o grafo S subjacente de G (em que todas as arestas de G estão presentes e tratadas como não-direcionadas)
2- realizar uma busca em profundidade a partir de um v (vértice) qualquer
3- se todos os vértices forem visitados, o grafo é Fracamente Conexo.

▷ **Semi-Fortemente conexo**: Quando, para qualquer par de vértices (u, v) existe um caminho unidirecional (de u p/ v ou de v p/ u). Podemos identificá-lo com os passos:

1- aplicamos uma busca ~~a partir~~ de um vértice. Se todos os vértices forem alcançáveis, ~~o grafo~~ é semi-fortemente conexo

★ um exemplo:



existe:
$d \to c$ ab db
$a \to c$ $d \to a$ ac dc
$b \to a$ $d \to b$ ad cb ✗

não é semi. fort. conexo.

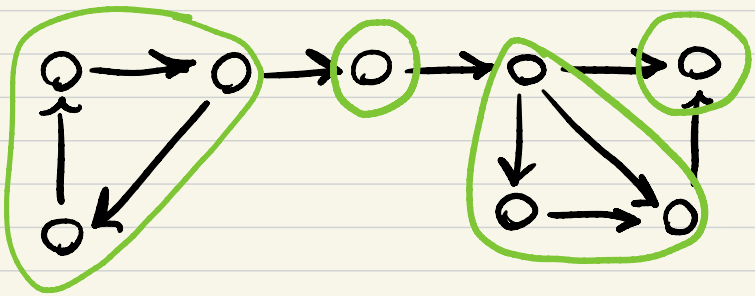
▷ **Fortemente conexo**: Quando existe um caminho em ambos os direções para qualquer par de vértices. Identificamos um grafo por meio do algoritmo Kosaraju com algumas adições, da seguinte forma:

1- realizamos uma DFS no grafo marcando os tempos de início e fim da visitação.
2- geramos o grafo transposto T de G.
3- aplicamos uma nova DFS a partir de T na ordem não-crescente dos tempos de término.
4- se todo o grafo for visitado numa única DFS em T, o grafo é fortemente conexo.

→ componentes fortemente conexos.

Kosaraju (SCC)

↳ algoritmo usado para encontrar componentes fortemente conexos em um grafo (e sua excentricidade)



Conjunto máximo de vértices tal que p/ todo par de vertice U e V, existe um caminho de U p/ V e de V p/ U.

↳ Passo a passo do algoritmo:

1- busca em profundidade (DFS) em G(V,E) gravando os tempos de início e fim da visitação

2- gerar o grafo transposto T de G.

3- na ordem não-crescente (inversa) dos tempos de término, realizar uma busca em profundidade no grafo T.

4- para cada conjunto de vértices visitados em T, marque-o como um componente fortemente conexo em G.

→ (caminho mínimo)

Algoritmos de Shortest-Path

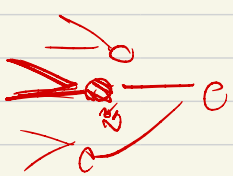
↳ algoritmos para encontrar o menor caminho em um grafo.

Dijkstra: Encontra o menor caminho de um vértice para todos os outros.

↳ Não funciona com arestas negativas (já que o Dijkstra não revisita nós já processados, ele não reconsidera que exista um caminho mais curto através de uma aresta negativa, pois já marcou o nó como finalizado quando encontrou o caminho mais curto até ele.

↳ É um algoritmo de busca gulosa (greedy algorithm), já que ele realiza uma decisão local repetidamente com objetivo de encontrar a melhor solução. → (e irreversível)

↳ Algoritmo:



```
Dijkstra(G, c):
    ∀ v ∈ V → d[v] = ∞
    d[u] = 0                          (vértice sendo processado)
    visitados = {u}
    while V ≠ visitados:              (seleciona, dentre todos os dists, a menor.)
        v* = arg min {d[w]} para v ∈ V \ visitados   (para todos os nós que ainda não foram visitados)
        visitados = visitados ∪ {v*}
        (e atribui a v*)
        ∀ v ∈ V →  d[v] = min(d[v*] + w(v*, v), d[v])
```

adicionamos o nó com a menor dist. acumulada aos visitados

atualiza a distância de cada vizinho só se a nova dist. for menor que a dist. já registrada.

$$d[v] = \min\left(d[v^*] + w(v^*, v),\ d[v]\right)$$

- $d[v^*] + w(v^*, v)$: distância de u até v^* + o peso da aresta (v^*, v)
- $w(v^*, v)$: peso
- $d[v]$: dist. acumulada de u até v

↳ Passo a Passo:

1- inicialize as distâncias como ∞ e a do nó inicial como 0, e crie o conjunto de nó visitados vazio.

2- entre todos os nós ainda não visitados, escolha o nó com a menor distância acumulada e atribuimos à variável v^*. Atribuimos v^* aos visitados.

3- para cada vizinho de v^*, calcule a nova distância potencial passando por v^*. Se essa nova distância for menor que a distância registrada anteriormente para o vizinho, atualize a distância do vizinho

4- repita o passo de escolher o vértice de menor distância acumulada até que não existam mais nós não visitados

Definição da Eva Tardos

Dijkstra's Algorithm (G, ℓ)
Let S be the set of explored nodes
    For each $u \in S$, we store a distance $d(u)$
Initially $S = \{s\}$ and $d(s) = 0$
While $S \neq V$
    Select a node $v \notin S$ with at least one edge from S for which
        $d'(v) = \min_{e=(u,v):u \in S} d(u) + \ell_e$ is as small as possible
    Add v to S and define $d(v) = d'(v)$
EndWhile

Bellman-Ford: Pode ser implementado em grafos com pesos negativos (só não com ciclos negativos, ou seja, ciclos em que a soma de seus pesos é negativa).

↳ Tem uma complexidade maior → enquanto o DijKstra usa uma fila de prioridade p/ selecionar os v a serem relaxados, o Bellman and Ford simplesmente relaxa todos os custos

↳ $O(V \times E)$

↳ Algoritmo:

```
Bellman_Ford(G, pesos):
    ∀ v ∈ V → d[v] = ∞
    visitados = {}
    d[u] = 0
    FOR EACH v in V:
        ∀ aresta = (e, v) ∈ E
            se d[v] > d[e] + peso(e,v)
                d[v] ← d[e] + peso(e,v)
                v* ← e
```

(relaxamento)

se a dist. atual for maior que a soma da dist d[e] + o peso da aresta, encontramos um caminho + curto para v passando por e.

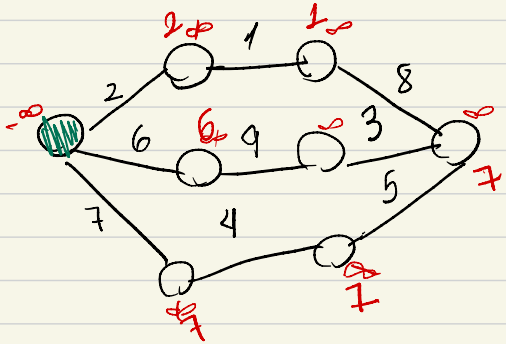
não escolhe a menor p/ relaxar, relaxa todas

↳ atualiza o predecessor

```
procedure shortest-paths(G, l, s)
Input:    Directed graph G = (V, E);
          edge lengths {l_e : e ∈ E} with no negative cycles;
          vertex s ∈ V
Output:   For all vertices u reachable from s, dist(u) is set
          to the distance from s to u.

for all u ∈ V:
    dist(u) = ∞
    prev(u) = nil

dist(s) = 0
repeat |V| − 1 times:
    for all e ∈ E:
        update(e)


procedure update((u, v) ∈ E)
dist(v) = min{dist(v), dist(u) + l(u, v)}
```

Algoritmos de Fluxo:

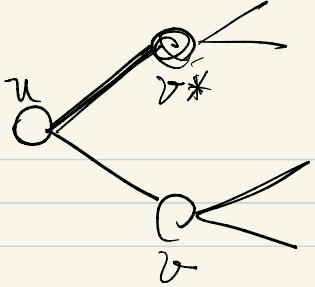
Min-max: encontra o caminho que maximiza o fluxo enquanto minimiza os gargalos. (Menor dos maiores)



Max-min: encontra o caminho que minimiza os fluxo máximo. (Maior dos menores) → maximiza os menores pesos de aresta ao longo do caminho.

```
Max-Min ( G(V,E), (G, w)      ← par dos custos
    F ← { }
    valores ← ∞ p/ a origem , -∞ p/ os demais
    F ← F ∪ { origem , valores(origem) }
    while F ≠ E:
        u ← extraiMaiorValor(F)
        for each v de u:
            menor_custo ← min( valores[u], W(u,v))
            se menor_custo > valores[v]:
                valores[v] ← menor_custo
                F ← F ∪ { v, valores[v] }
```

Max-Min



Dijkstra (G, l):
$\forall v \in V \rightarrow d[v] = \infty$
$d[u] = 0$
visitados = {u}
while V ≠ visitados:
$v^* = \arg\min \{d[w]\}$ para $v \in V \setminus$ visitados
visitados = visitados $\cup \{v^*\}$
$\forall v \in V \rightarrow d[v] = \min(d[v], d[v^*] + w(v^*, v))$

nuevo v

$\max(d[v], \min(d[w], w(u,v))$

Algoritmo Widest Path

→ Max-Min

↳ Usado p/ encontrar o caminho máximo de v a u em um grafo não direcionado $G = (V, E)$.

↳ é uma adaptação do Dijkstra

↳ Passo a Passo:

1- inicializamos um conjunto $P = V$ (conjunto de V que ainda não foram visitados) e um vetor W que aprenta $+\infty$ para V (ponto + forte possível) e $-\infty$ para todo vertice pertencente a P/V (ainda não sabemos o melhor caminho p/ eles).

2- iniciamos uma busca em largura partindo de V usando uma fila de prioridade ordenada pelo vetor W de modo que o vértice com peso mais alto em W seja removido primeiro (estamos sempre expandindo o caminho que tem a maior largura possível).

3- enquanto $P \neq \{\}$
- ↳ removemos o vértice w c/ maior valor de W da fila dp. e de P.
- ↳ ∀ vizinho u de w em P, altere o valor de u para $\max(\min(W[w], l(w,u)), W[u])$, e adicione u na fila de prioridade.

$\max(\min(W[w], l(w,u)), W[u])$

$\min(W[w], l(w,u))$ → armazena o valor máximo entre o caminho que estamos expandindo e o valor de $W[u]$,

= pegado { calcula o menor valor entre:
- $W[w]$ = largura do caminho + largo até agora p/ w.
- $l(w,u)$ = o peso da aresta entre w e o vizinho u

Minimum Spanning Trees:

↳ algoritmos p/ gerar a MST (uma árvore (sem ciclos) que conecta todos os vértices do grafo original com o menor peso total possível).

(soma, ou seja, dos custos das arestas)

Prim: análise local (verifica as arestas não visitadas vindo das visitadas).

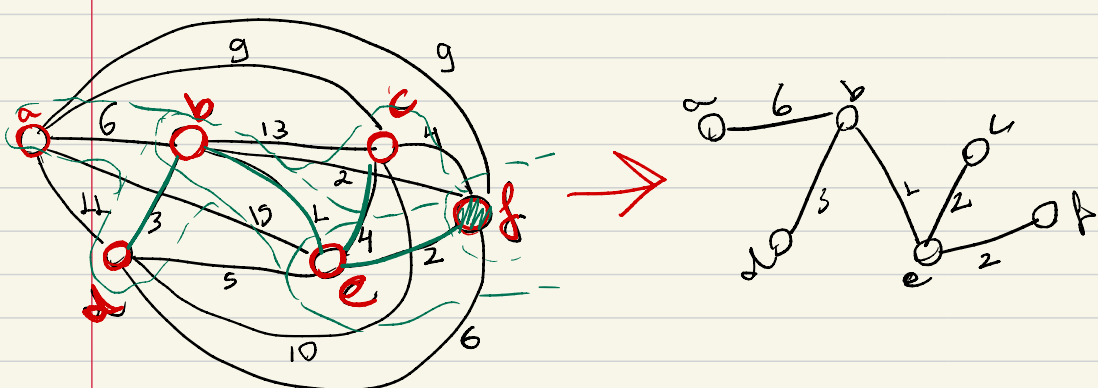
↳ Passo a Passo: → Grafo (V, E)

1- começa em um vértice do conjunto S (de vértices), que pertence ao V, que inicialmente contém v (um v aleatório inicial que pertence a V).

2- para adicionarmos uma aresta $\{w, u\}$ ao conjunto Solução, w deve pertencer a S, e u deve pertencer a $V \setminus S$, de modo que essa aresta $\{w, u\}$ pertencente a E deve ser a aresta de menor peso no conjunto de corte de S.

se esses requisitos forem cumpridos, adicionamos $\{w, u\}$ ao conjunto Solução e u à S.

3- o algoritmo se encerra quando $|S| = |V|$



Kruskall: análise global (analisa o grafo todo). Ao invés de um só conjunto S, temos vários.

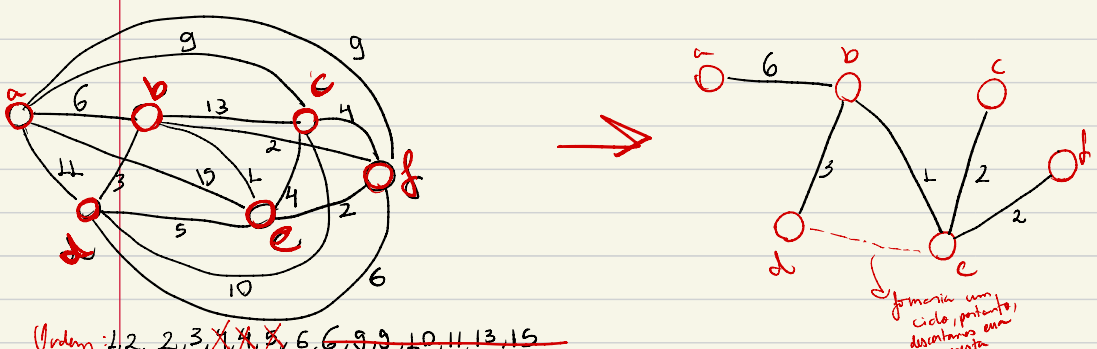
↳ Passo a Passo: (Grafo conexo, não-direcionado, ponderado $G(V,E)$ e custos R^+)

1- ordenamos todas as arestas por peso em ordem não-decrescente

2- criamos um grafo nulo T com todos $v \in V$.

3- seguindo a ordem não-decrescente de pesos, adicionamos as arestas de G em T somente se, ao adicionar uma aresta, o grafo T continue acíclico (não pode gerar ciclos em T).
↳ se os dois pertencem ao mesmo componente conexo

4- assim que o nº de arestas de T for igual a $|V|-1$, temos a resposta da MST.



Ordem: 1, 2, 2, 3, ~~4, 4, 5~~, 6, ~~6, 9, 9, 10, 11, 13, 15~~
↳ já chegamos em $|V|-1$ arestas

Cut Property:

(4.17) *Assume that all edge costs are distinct. Let S be any subset of nodes that is neither empty nor equal to all of V, and let edge $e = (v, w)$ be the minimum-cost edge with one end in S and the other in $V - S$. Then every minimum spanning tree contains the edge e.*

Algoritmo de Johnson:

↳ técnica p/ encontrar o menor caminho de n para n vértices do Grafo.

↳ Passo a Passo:

1- transformamos o grafo adicionando um novo vértice S que tenha uma aresta de peso 0 p/ todo v de V.

2- aplicamos o Bellman-Ford nesse grafo alterado a partir do vértice S p/ calcular a menor distância d[v] de s p/ cada v de V.

3- se o Bellman-Ford encontrar ciclos negativos, o algoritmo se encerra.

4- com os valores d[v], ajustamos o peso original de cada aresta {u, v} no G original com a fórmula → peso'(u,v) = peso(u,v) + d[u] - d[v]. Essas alterações garantem que todos os pesos sejam não-negativos (p/ usar o Dijkstra).

5- aplicamos o Dijkstra p/ cada v em V do novo Grafo.

6- p/ obter as distâncias corretas no grafo original, revertemos a reponderação dos pesos com $d[u,v] = d'[u,v] - d[u] + d[v]$

Maior caminho (em nº de arestas)

a) $G(V, E) \rightarrow (G, W)$, $W: E \rightarrow \{1\}$
b) $W' = -W$
2) aplicamos o Dijkstra seguindo uma ordem
3) resposta do Dijkstra é "invertida".

- acha a base
- começamos no v que terá o maior fecho transitivo.

Excentricidade:

semelhante ao Dijkstra, mas sem utilizar pesos (contando os vértices).

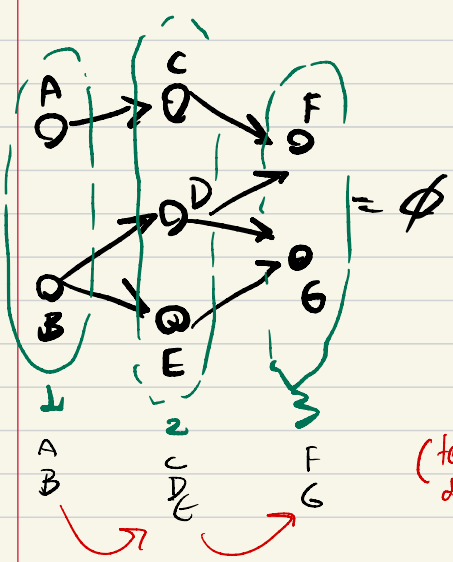
```
Excentricidade(G, u):
    d[u] = 0
    S = V
    for each v ∈ V
        d[v] = ∞
    while S ≠ ∅
        v = extrai Min(S)
        for each v* ∈ vizinhos(v)
            d[v*] = Min(d[v+1], d[v*])
    excentricidade = max(d[v] for v ∈ V)
```

Caminho Crítico

↳ usado p/ alcançar o vértice mais distante no menor tempo possível.

vértices c/ grau de entrada 0

- identificamos a base
- removemos a base
- fazemos isso até o grafo ficar vazio (enquanto tiver conjuntos de bases diferentes adiciona-os em um caminho).
- isso garante a ordem topológica.



B A D C E G F

(tem vários jeitos de encontrar ele)